Not built — scope honesty

I spent roughly **8 hours**. What I deliberately left out, and why.

Cut on purpose

- **UI.** No customer form, shop dashboard, or admin console. The rubric scores architecture and failure handling, not UI. The endpoints + `make replay` + the per-message status lookup demonstrate every behavior. A thin read-only page would be the first UI I'd add.
- **Real customer notifications.** The "customer-facing message" is returned by the status endpoint; I don't actually send email/SMS. In prod this is a notifier that consumes work-order state changes.
- **Cancellation handling** (sample m-1038). Detected as `intent=cancellation` and parked as `NEEDS_REVIEW` for a human — I do not auto-cancel a prior appointment. Doing it safely needs linking to the original work order and an audit trail.
- **User accounts / OAuth.** Auth is a per-shop API key, which doubles as the tenant. Per-user login, roles, and key rotation are out of scope.
- **Follow-up beyond one round.** The schema asks exactly one question (as the brief says). Multi-turn clarification isn't modelled.
- **Date resolution.** "Saturday", "tomorrow" are captured as text / best-effort ISO, not resolved to concrete dates against shop calendars or availability.

Known rough edges

- The model occasionally **assumes** rather than asks (e.g. "Brakes squeal." → both brakes). Tunable via the prompt; I chose not to over-constrain it.
- **Spend cap overshoot** of one call (pre-check, not reservation) — documented tradeoff in the architecture note.
- A worker that crashes mid-job leaves it `IN_PROGRESS`; there's no reaper to requeue stuck jobs. Fine for a prototype; needed in prod.

What I'd build next (in order)

1. Swap the outbox for **Cloud Tasks** (retry/backoff/dedup/rate-limit for free).
2. A **stuck-job reaper** + dead-letter visibility.
3. **Notifier** service for real customer comms.
4. A thin **shop read UI**.
5. **Golden-set eval** for extraction quality (precision/recall on the catalog mapping), run in CI against a frozen sample set.

What breaks first at 100 shops

- **Per-shop polling.** The worker loops over shops; at 100 that's wasteful and latency grows. Fix: push-based Cloud Tasks (per-shop queues give natural isolation and rate-limiting), so work is dispatched, not polled.
- **One shared Postgres / one `app.shop_id` model** still holds at 100 shops, but noisy-neighbour and connection limits appear — add per-tenant rate limits and pooling (PgBouncer), and revisit schema-per-tenant only if isolation needs harden.
- **Spend accounting contention** if many workers process one shop concurrently — move from pre-check to an atomic reservation on the ledger row.
- **Catalog assumed global.** At 100 shops, pricing likely diverges per shop/region; the catalog becomes per-tenant and versioned.